

Day 35 - Web Scraping Using Python

Introduction to Web Scraping

What is Web Scraping?

Web Scraping is the process of automatically extracting data from websites using Python programs.

Instead of manually copying information from a website, a scraper collects the required information and stores it for further processing.

Web Scraping is widely used in Data Science, Data Analytics, Artificial Intelligence, Machine Learning, Automation, and Business Intelligence.

Why Use Web Scraping?

Web Scraping helps us to:

- Collect website data automatically
- Save time and manual effort
- Analyze large amounts of information
- Monitor changes on websites
- Store website data for future analysis

Real-Time Applications

- Product Price Tracking
- News Aggregation
- Job Portal Data Collection
- Weather Information Collection
- Stock Market Analysis
- Hotel and Flight Price Monitoring
- Research and Data Analysis
- Lead Generation
- Business Intelligence
- Competitor Analysis

Web Scraping Process

```
None
Website
↓
Requests Module
↓
HTML Source Code
↓
BeautifulSoup
↓
Extract Required Data
↓
Store in CSV / Excel / Database
```

What is HTML?

HTML (HyperText Markup Language) is the standard language used to design web pages.

Every webpage is made up of HTML tags such as headings, paragraphs, links, images, tables, forms, etc.

Example:

```
HTML
<html>
<head>
  <title>Codegnan</title>
</head>
<body>
  <h1>Welcome to Codegnan</h1>
  <p>Python Full Stack Course</p>
  <a href="https://codegnan.com">
    Visit Codegnan
  </a>
</body>
</html>
```

BeautifulSoup reads these HTML tags and allows us to extract useful information.

Requests Module

Definition

The **Requests** module is one of the most popular Python libraries used to send HTTP requests to websites. It helps us retrieve the HTML source code of a webpage.

Installation

```
Shell  
pip install requests
```

Import

```
Python  
import requests
```

Sending a GET Request

The `get()` method sends an HTTP GET request to a website.

Syntax

```
Python  
requests.get(url)
```

Example

```
Python  
import requests  
url = "https://codegnan.com"  
response = requests.get(url)  
print(response)
```

Output

None

```
<Response [200]>
```

Response Object

The `get()` method returns a **Response** object. The Response object contains information about the webpage returned by the server.

Python

```
response = requests.get(url)
```

Important Response Attributes

Attribute	Description
<code>status_code</code>	Returns the HTTP response status
<code>text</code>	Returns HTML source code as a string
<code>content</code>	Returns HTML source code as bytes
<code>headers</code>	Returns website response headers
<code>url</code>	Returns the final URL
<code>cookies</code>	Returns cookies sent by the server
<code>encoding</code>	Returns webpage encoding
<code>elapsed</code>	Returns response time

status_code

Returns the HTTP status code.

Syntax

Python

```
response.status_code
```

Example

Python

```
import requests  
response = requests.get("https://codegnan.com")  
print(response.status_code)
```

Output

None

200

text

Returns the HTML source code of the webpage as a string.

Syntax

Python

```
response.text
```

Example

Python

```
import requests  
response = requests.get("https://codegnan.com")  
print(response.text[:500])
```

Output

HTML

```
<!DOCTYPE html>  
<html>  
<head>
```

content

Returns the HTML source code as bytes.

Useful while downloading images, PDFs, and other files.

Syntax

```
Python  
response.content
```

Example

```
Python  
import requests  
response = requests.get("https://codegnan.com")  
print(response.content)
```

Output

```
None  
b'<!DOCTYPE html>...'
```

headers

Returns all HTTP response headers.

Syntax

```
Python  
response.headers
```

Example

```
Python  
import requests  
response = requests.get("https://codegnan.com")
```

```
print(response.headers)
```

url

Returns the final webpage URL after redirects.

Syntax

```
Python  
response.url
```

Example

```
Python  
import requests  
response = requests.get("https://codegnan.com")  
print(response.url)
```

Output

```
None  
https://codegnan.com/
```

cookies

Returns cookies received from the website.

Syntax

```
Python  
response.cookies
```

Example

Python

```
import requests
response = requests.get("https://codegnan.com")
print(response.cookies)
```

encoding

Returns the encoding used by the webpage.

Syntax

Python

```
response.encoding
```

Example

Python

```
import requests
response = requests.get("https://codegnan.com")
print(response.encoding)
```

Output

None

UTF-8

elapsed

Returns the time taken by the server to respond.

Syntax

Python

```
response.elapsed
```

Example

Python

```
import requests
response = requests.get("https://codegnan.com")
print(response.elapsed)
```

Common HTTP Status Codes

Code	Meaning
200	Success
201	Resource Created
301	Moved Permanently
302	Temporary Redirect
400	Bad Request
401	Unauthorized
403	Forbidden
404	Page Not Found
405	Method Not Allowed
500	Internal Server Error
502	Bad Gateway
503	Service Unavailable

Example: Checking Website Status

Python

```
import requests
url = "https://codegnan.com"
response = requests.get(url)
if response.status_code == 200:
    print("Website is accessible.")
```

```
else:  
    print("Unable to access the website.")
```

Output

```
None  
Website is accessible.
```

Example: Printing the HTML Source Code

```
Python  
import requests  
url = "https://codegnan.com"  
response = requests.get(url)  
print(response.text[:1000])
```

BeautifulSoup

What is BeautifulSoup?

BeautifulSoup is a Python library used to parse HTML and XML documents.

It converts HTML source code into a searchable object, making it easy to navigate, search, and extract data from web pages.

BeautifulSoup works together with the **Requests** module.

- **Requests** download the webpage.
- **BeautifulSoup** extracts the required information from the downloaded HTML.

Installation

```
Shell  
pip install beautifulsoup4
```

Import

Python

```
from bs4 import BeautifulSoup
```

Creating a BeautifulSoup Object

Syntax

Python

```
BeautifulSoup(html_content, "html.parser")
```

Example

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
```

Now the webpage HTML has been converted into a BeautifulSoup object.

BeautifulSoup Object

The soup object allows us to search HTML elements such as

- Title
- Headings
- Paragraphs
- Links
- Images
- Tables
- Forms

Extract Website Title

Method 1

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
print(soup.title)
```

Sample Output

HTML

```
<title>Codegnan | Job Ready Courses</title>
```

Method 2

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
print(soup.title.text)
```

Sample Output

None

```
Codegnan | Job Ready Courses
```

Extract First Heading

Python

```
import requests
from bs4 import BeautifulSoup
```

```
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
print(soup.h1.text)
```

Extract First Paragraph

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
print(soup.p.text)
```

find()

Purpose

Returns the **first matching HTML tag**.

Syntax

Python

```
soup.find(tag_name)
```

Example

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
heading = soup.find("h1")
```

```
print(heading.text)
```

Example

```
Python
paragraph = soup.find("p")
print(paragraph.text)
```

find_all()

Purpose

Returns **all matching HTML tags** as a list.

Syntax

```
Python
soup.find_all(tag_name)
```

Example

```
Python
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
links = soup.find_all("a")
print(links)
```

Example

```
Python
headings = soup.find_all("h2")
for heading in headings:
```

```
print(heading.text)
```

text

Purpose

Returns only the text inside an HTML tag.

Syntax

```
Python  
tag.text
```

Example

```
Python  
import requests  
from bs4 import BeautifulSoup  
url = "https://codegnan.com"  
response = requests.get(url)  
soup = BeautifulSoup(response.text, "html.parser")  
heading = soup.find("h1")  
print(heading.text)
```

Example

```
Python  
paragraph = soup.find("p")  
print(paragraph.text)
```

Extract Hyperlinks

Links are stored inside the href attribute.

Example HTML

HTML

```
<a href="https://codegnan.com">  
Visit Codegnan  
</a>
```

Extract Attributes

Method 1

Python

```
link = soup.find("a")  
print(link["href"])
```

Method 2

Python

```
link = soup.find("a")  
print(link.get("href"))
```

get()

Purpose

Returns the value of an HTML attribute.

Syntax

Python

```
tag.get(attribute_name)
```

Example

Python

```
import requests  
from bs4 import BeautifulSoup
```

```
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
link = soup.find("a")
print(link.get("href"))
```

Extract All Headings

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
headings = soup.find_all(["h1", "h2", "h3"])
for heading in headings:
    print(heading.text)
```

Extract All Hyperlinks

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
links = soup.find_all("a")
for link in links:
    print(link.get("href"))
```

Extract Images

Image URLs are stored inside the src attribute.

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
images = soup.find_all("img")
for image in images:
    print(image.get("src"))
```

Extract Tables

Tables are represented using the <table> tag.

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
tables = soup.find_all("table")
print(tables)
```

Tables are commonly used to extract:

- Employee Records
- Student Data
- Product Lists
- Stock Market Data
- Cricket Scorecards
- Reports

Real Example – Extract Website Title

Python

```
import requests
from bs4 import BeautifulSoup

url = "https://codegnan.com"
```

```
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
title = soup.find("title")
print(title.text)
```

Real Example – Extract All Links

Python

```
import requests
from bs4 import BeautifulSoup

url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
links = soup.find_all("a")
for link in links:
    print(link.get("href"))
```

Common BeautifulSoup Methods

Method	Description
find()	Returns the first matching tag
find_all()	Returns all matching tags
text	Extracts text from a tag
get()	Extracts an attribute value
select()	Returns all matching elements using CSS selectors
select_one()	Returns the first matching element using CSS selectors
parent	Returns the parent tag
children	Returns all child tags
prettify()	Displays formatted HTML

select()

Purpose

`select()` searches HTML elements using CSS selectors and returns all matching elements.

Syntax

Python

```
soup.select(css_selector)
```

Example

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
links = soup.select("a")
for link in links:
    print(link.get("href"))
```

select_one()

Purpose

Returns only the first matching element using a CSS selector.

Syntax

Python

```
soup.select_one(css_selector)
```

Example

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
title = soup.select_one("title")
print(title.text)
```

parent

Purpose

Returns the parent element of a tag.

Syntax

Python

```
tag.parent
```

Example

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
title = soup.find("title")
print(title.parent.name)
```

Sample Output

None

head

children

Purpose

Returns all child elements of a tag.

Syntax

```
Python  
tag.children
```

Example

```
Python  
import requests  
from bs4 import BeautifulSoup  
url = "https://codegnan.com"  
response = requests.get(url)  
soup = BeautifulSoup(response.text, "html.parser")  
body = soup.find("body")  
for child in body.children:  
    print(child)
```

prettify()

Purpose

Displays HTML in a properly formatted and indented structure.

Syntax

```
Python  
soup.prettify()
```

Example

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
print(soup.prettify())
```

CSS Selectors

CSS Selectors help us locate HTML elements more efficiently.

Select by Tag

Syntax

Python

```
soup.select("tag_name")
```

Example

Python

```
headings = soup.select("h2")
for heading in headings:
    print(heading.text)
```

Select by Class

Syntax

Python

```
soup.select(".class_name")
```

Example

```
Python
buttons = soup.select(".btn")
for button in buttons:
    print(button.text)
```

Select by ID

Syntax

```
Python
soup.select("#id_name")
```

Example

```
Python
header = soup.select("#header")
print(header)
```

Select Nested Elements

Syntax

```
Python
soup.select("parent child")
```

Example

```
Python
links = soup.select("nav a")
for link in links:
    print(link.text)
```

Extract Website Title

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
print(soup.title.text)
```

Extract All Hyperlinks

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
links = soup.find_all("a")
for link in links:
    print(link.get("href"))
```

Extract All Images

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
images = soup.find_all("img")
for image in images:
    print(image.get("src"))
```

Extract All Headings

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
for heading in soup.find_all(["h1", "h2", "h3"]):
    print(heading.text.strip())
```

Count Hyperlinks

Python

```
import requests
from bs4 import BeautifulSoup
url = "https://codegnan.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
links = soup.find_all("a")
print("Total Links:", len(links))
```

Note: Dynamic Websites

Not all websites can be scraped using only **Requests** and **BeautifulSoup**. Modern websites built with **React**, **Angular**, or **Vue** load their content using JavaScript after the webpage opens. One such example is the **Codegnan Placement Portal**.

Python

```
url = "https://placements.codegnan.com"
```

If you execute:

Python

```
import requests
response = requests.get("https://placements.codegnan.com")
print(response.text)
```

You may receive HTML similar to:

HTML

```
<div id="root"></div>
<script src="/assets/index.js"></script>
```

The actual job listings are **not present** in the HTML returned by requests.

This happens because the webpage loads the data dynamically using JavaScript.

In such cases, we should use:

- Selenium
- Playwright
- Backend APIs (if available)

instead of only requests and BeautifulSoup.

This is an important limitation of basic web scraping.

Colab Link: [🔗 WebScraping.ipynb](#)